OO Analysis & Design Project 2026 Instructions

- There are two sections to this project.
- **Section A** requires you to read a specification for a furniture business and create several analysis and design models.
- **Section B** requires you to answer some design pattern related questions. Any referenced material must be cited.
- All answer material must be included in the report documents.
- Please ensure that all questions that require a descriptive answer are written in your own words.
- Copying and pasting text is not acceptable and you will lose marks if your text is found as copied text. Include references to any material cited.
- You will be required to submit solutions in electronic format before **midnight of Sunday April 26th.**
- You must submit **three** files as part of your project submission, 1-word files [part A and Part B], and 2 Visual Paradigm files[Part A and Part B]. [**use VP 17.2 only**]. The word documents will contain answers to all questions. Any models created in Visual Paradigm should be copied as images into report documents.
- Include a Cover page and a table of contents.
- A student declaration of the following nature should be included at the back of the project. Plagiarized material **will not** be allocated a mark.
- You will be expected to attend labs during which time I can monitor your progress.

(** I will not correct any student's work who does not attend labs**)

The project is completed in pairs. You need to group with one other student from your Lab Group and enter your group details in the sheet here [Untitled spreadsheet - Google Sheets]

Note: Attendance at the labs is mandatory during the duration of the project. There will be 10% of overall project mark assigned to full attendance.

Student declaration

I can confirm the following details:

Student ID/Registration number: _____

Name: _____

Module Name: OO Analysis and Design

Lab Lecturer: Nasir Ahmad

Due Date: April 26, 2026 @midnight

Part A 70%

Use the specification provided for Oak Valley Furniture Company to carry out the tasks below.

Requirements Specification

Oak Valley is a furniture business with multiple stores across Ireland. It sells several different furniture products. These products are grouped into several different product lines. A product line is noted by a product line id number and a name for the product line. One product line may be a group of any number of products. A product must belong to exactly 1 product line. There may be new products available that have not been ordered by any customer yet. Some products appear on many customer orders. Orders can be placed directly by customers or by salespeople on their behalf in a store. Each order placed contains an order id number, the quantity of any selected products, the date the order was placed. An order may consist of more than one product. Each order is for exactly one customer. The quantity of a product in stock may vary from store to store. Several warehouses are used to store surplus stock. If an order is made online for a product, then a check is made to determine if it is available in the region a customer is located in. A person must register their details before placing an order. The following information is maintained on customers: their name (first and last name), address, mobile number, email address. The payment methods used by oak Valley are card or PayPal. An order can be cancelled within 24 hours of its placement. A customer will be informed of the expected delivery date of an order. When an order is delivered it must be signed for and dated.

Oak valley has established sales regions, and each customer does business in one of these sales regions. A store is in exactly one sales region. A sales region (noted by an id number and region name) may have any number of customers. Oak valley has several salespersons, each of whom are assigned to exactly 1 store. Some of the larger stores have more than one salesperson assigned to them. For each salesperson, the salesperson identifying number, name (first and last) and mobile phone number are maintained. For orders submitted in a store, Oak valley wants to keep track of which salesperson handled it for commission purposes. Note a delivery of an order is handled by a driver working in the same region that the order is to be delivered to. The delivery address may be different from the customer's address for a particular order, but both are in the same region.

Over 35% of addresses in Ireland share their address with at least one other property, making it difficult for those delivering goods and services to accurately identify the location of an address.

Eircode has been designed specifically to overcome this problem. Unlike other countries where postcodes define a cluster or group of addresses, a unique Eircode is assigned to each residential and business address. Oak Valley wants to exploit the benefits of using Eircode's for deliveries.

A customer must supply an Eircode to the delivery address of an order.

Oak Valley products are supplied by one or more vendors. Each vendor may supply more than one product. The price of a given product is dependent on the quantity purchased from the vendor at a given time. The retail price of a product can be discounted at certain times of the year.

Oak valley owns a fleet of delivery vehicles which work in the established sales regions. It always wants to be able to monitor its fleet of courier vehicles, noting the GPS location, the driver, and the orders in his/her care. The vehicle details recorded are make, model, registration number and current mileage. Each driver is assigned to work in a particular sales region.

The company wants to use GPS for courier tracking.

1. *Create a Domain Model for Oak Valley.* [15]

2. *Create system sequence diagrams to model the following features:* [10]
[Create a use case diagram to model features. System sequence diagrams should be added as sub diagrams for each use case.]

   i. *Find the delivery address of an order.*

   ii. *Find the status of an order.*

   iii. *Find the journey time for a particular delivery schedule.*

   iv. *Find the total cost of an order for a particular customer.*

   v. *Find the current location of a driver who is out on a delivery of a customer order.*

3. ***Identify** and **list** operations signatures required to complete each feature. Assign each operation to appropriate class.* [10]

4. *Create sequence diagrams to realize the happy scenario **for each** of the above features. Use operations specified in [3] to create messages between object lifelines. [sequence diagrams should be added as sub diagrams of Use cases]* [10]

5. *Create a Design Class Diagram to support the sequence diagrams created in 4. Your design class diagram should contain at least one controller class.* [10]

6. *Extend design Class Diagram to allow for the following additional requirement.* [5]

   i. *A customer would like to return a product that was ordered and delivered. The date and reason for return needs to be tracked.*

Notes:

1. Your VP project file should contain the following two models.

   Analysis Model: Domain Model, System sequence diagrams

   Design Model: Design Class diagram and Sequence diagrams.

2. Please note any assumptions you make when making an analysis/design decision.

   This is essential as specification is incomplete.

3. Include one Singleton class in your design model.

Part B 30 %

Answer any two questions.[Include references]

Question 1

Review the code provided below to answer the following question. [10]

The **LoanApprovalHandler** class is responsible for approving two different types of loans. .The code violates the **Open closed principle** [OCP].

i. Draw a class diagram to model the code.
ii. Explain why the code violates the open closed principle.
iii. Rewrite the code so it does not violate this design principle.
iv. Draw a class diagram to model revised code.

```
public class LoanApprovalHandler
{
  public void approvePersonalLoan (PersonalLoanValidator validator)
  {
    if ( validator.isValid())
    {
      //Process the loan.
    }
  }
  public void approveVehicleLoan ( VehicleLoanValidator validator )
  {
    if ( validator.isValid())
    {
      //Process the loan.
    }
  }
  // Method for approving other loans.
}

public class PersonalLoanValidator
{
  public boolean isValid()
  {
    //Validation logic
  }
}
public class VehicleLoanValidator
{
  public boolean isValid()
```

```
{
  //Validation logic }}
```

Question 2 [10]

Create a design class diagram to model a home entertainment system and the ability for a user to be able to use a single remote control to control multiple digital devices such as a TV or sound system etc.

Explain your design

Create a Remote-Control class with 3 methods in it. One method is for choosing the device to connect the remote control to (make this a static method), and the other two methods are for turning that particular device on or off. Create three other classes – Projector, TV, SurroundSoundSystem. All three of these devices should follow a consistent interface so that the same remote can be used to turn the devices on or off. Remember, you want to implement this in such a way that if more devices need to be controlled by the same remote in the future, that can be easily done. We don't want to keep modifying the internals of the remote-control class to be able to support more devices in the future. The last class you can create is the Person class. This will be the user of the home entertainment system. You can define a main method in this class to test out your code. Initialize the devices here and have the remote connect to that device to turn it on or off. Indicate how a new device could be added to home entertainment system without breaking existing code.

Question 3 [10]

i. Explain the meaning of the **Façade design** pattern using the code below.

ii. Reverse engineer code into a **UML class** diagram.

iii. Draw a sequence diagram to illustrate **main** method.

```
class CPU {
    public void processData() { }
}
```

```
class Memory {
    public void load() { }
}

class HardDrive {
    public void readData() { }
}

class Computer {
    private CPU cpu;
    private Memory memory;
    private HardDrive hardDrive;

    public Computer() {
        this.cpu = new CPU();
        this.memory = new Memory();
        this.hardDrive = new HardDrive();
    }

  public void run()
  { cpu.processData();
    memory.load();
     hardDrive.readData();
    }
}


class User {
    public static void
        main(String[] args) {
        Computer computer = new
        computer();
        computer.run();
    }
}
```